

Real-time messaging (WhatsApp) front: what to ask, what to design · endpoints before boxes · inbox for durability · ordering is policy

Anthropic questions this is: SD Q4, the 1:1 chat card (1point3acres, 11 discussions; Paul Grunchich's own opener: "one-to-one, text only, web only", everything else must be asked; drives into WebSocket gateways with a consistent hash, the Inbox table, presence TTL, Redis Pub/Sub vs Kafka, "now add a second device") · Tyler's WhatsApp form = the card's follow-ups (group, multi-device, presence) · **hellointerview** chat and messaging (33 reports) · **prachub** resilient chat, one-to-one chat · **hack2hire** Messenger, Slack-like chat. **Anthropic's extra probes:** the Kafka-vs-Redis metadata argument and Kafka internals; sharding by user vs conversation; the second device.

1 REQUIREMENTS 0–5 min · the five-question pack, then sentences

- **Ask:** group size cap (DM = 2; groups 256 / 1,000; above = channel)? media types (text only, or images/video: storage and throughput)? delete/edit semantics (for me, for everyone, within N min)? multi-device policy (phone + tablet at once)? presence accuracy (seconds or minutes; last_seen needed)?
- **A user can:** send and receive in DMs and groups · see sent / delivered / read · see who is online · scroll history · use several devices · get push when offline · create, rename, join, leave groups.
- **Technical challenges:** delivery <1 s p99 in region · never lose a message (durable before ack) · **availability over consistency: accept out-of-order arrival, render by timestamp** · offline and multi-device catch-up · group fan-out cost · presence without last_seen writes.
- **Anchors:** 100M DAU × 10 = 1B/day = 11.5k/s avg, 35k/s peak · 10M sockets ≈ 10 gateways · 200 B → 200 GB/day · 256-member group = 256 deliveries.

2 CORE ENTITIES 5–9 min · DM is a two-member conversation

- **User**(id, handle) · **Device**(id, user_id, platform, push_token)
- **Conversation**(id, type dm|group, name) · **ConversationMember**(conversation_id, user_id, role, joined_at, muted)
- **Message**(id monotonic per conversation, conversation_id, sender_id, client_msg_id, sender_ts, server_ts, body) — immutable
- **Receipt**(message_id, recipient_id, device_id, state sent|delivered|read, at) — per message, per recipient, per device
- **Presence**(user_id → online until TTL) · **Inbox**(recipient, device, message_id) — the durability backstop
- Not entities: the socket, the notification.

3 API 9–13 min · write these before any box

- **POST** /conversations {type, member_ids, name} · **GET** /conversations?since&limit · **POST|DELETE** /conversations/{id}/members/{uid} (leave detaches push) · **PATCH** /conversations/{id} {name}
- **POST** /conversations/{id}/messages {client_msg_id, body, sender_ts} → {message_id, server_ts}, idempotent · **GET** .../messages?since=message_id&limit
- **POST** /messages/{id}/receipts {state} · **GET** /messages/{id}/receipts
- **PATCH** /users/{id}/presence (heartbeat) · **GET** /users/{id}/presence · **WS** /ws frames: message, receipt, presence
- **Hot:** send, read, receipt, heartbeat. **Cold:** create, rename, add, leave. The list shows which services each flow touches.

4 DATA FLOW 13–20 min · one message, DM then group

- device → gateway (consistent hash of user id owns the socket) → message service persists (server_id, server_ts) and writes **one inbox row per recipient device** → publishes on each recipient's channel →
- owning gateway pushes; no socket → push notification via device token → device ACKs → inbox row deleted, **delivered** receipt fanned to the sender → on open, **read**.
- **Group:** same path N times; above ~100 members hand fan-out to workers off the send path. **Offline:** reconnect, GET since last id, ACK, rows cleared.
- **Ordering:** server id per conversation for storage; render by sender_ts, flag late arrivals; dedupe retransmits on client_msg_id.

5 HIGH-LEVEL DESIGN 20–33 min · the board

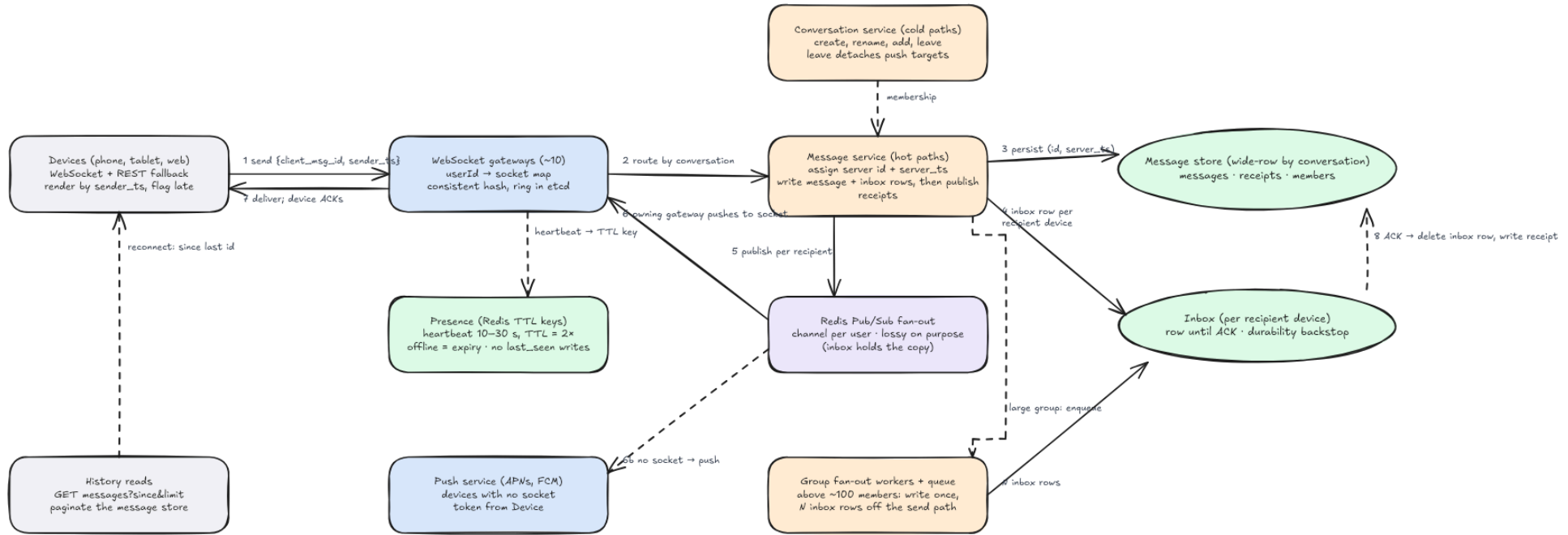
- Clients → L4 → WebSocket gateways (userId → socket, ring in etcd) → conversation service (cold) + message service (hot) → message store (wide-row by conversation) · inbox · presence (Redis TTL) · Redis Pub/Sub fan-out · fan-out workers + queue · push (APNs/FCM) · history reads paginated.
- **Kafka vs Redis, once:** Pub/Sub for fan-out because the inbox already guarantees durability; single-digit ms, no per-channel metadata. Kafka only if hours of replay is a hard requirement; topic-per-user ≈ 50 KB metadata each, 50 TB at 1B users.
- **Presence:** heartbeat 10–30 s, TTL 2×, offline = expiry; last_seen per heartbeat would be 20M writes/s.

6 DEEP DIVES 33–48 min · the stall first, then the card's probes

- **Ordering policy:** storage order = server id; display order = sender_ts clamped to server_ts ± window, late-arrival marker; retransmits dedupe on client_msg_id; no global order, ever.
- **Multi-device:** inbox and receipts keyed by device; delivered = any device has it, read = any device opened it (say the product choice); cap devices.
- **Group fan-out:** write once, inbox N times; workers above ~100; read receipts as counts.
- **Delivery guarantee:** at-least-once bus + client dedupe = exactly-once for the user; the inbox row is why the bus may be lossy.
- **Sharding:** many small conversations → by user id; few huge → by conversation id; hybrid above a threshold.
- **Failure:** gateway dies → ring slice reconnects, inboxes replay; store partition → writes fail closed for that conversation; push down → inbox holds. **Bow:** endpoints first; durable in store and inbox before anyone is told; bus fast and lossy on purpose; ordering is a client policy over server ids; groups are the same path with a worker doing the N.

The board at minute thirty

back: the answer, as it should look on the shared screen



Failure / probes: gateway dies → its ring slice reconnects elsewhere, inboxes replay. Bus drops a publish → inbox row still there, delivered on reconnect. Push provider down → inbox holds until the socket returns. Ordering: server id per conversation for storage; render by sender_ts, clamp to server_ts ± window, mark late arrivals; dedupe retransmits on client_msg_id. Global order is the trap.

Numbers to say: 1B msgs/day → 11.5k/s avg, 35k/s peak · 10M sockets = 10 gateways · 200 GB/day · delivery < 1 s p99 · presence TTL 2x a 10-30 s heartbeat; last_seen per heartbeat would be 20M writes/s · Redis Pub/Sub single-digit ms; Kafka topic-per-user = 50 KB metadata each, 50 TB at 1B users.

Multi-device: inbox and receipts keyed by device; delivered = any device has it; read = any device opened it (say the product choice). Group: write once, inbox N times, workers above ~100 members, read receipts as counts.

Legend

□ clients / history □ gateways, push □ services □ stores, presence □ fan-out bus

solid = the message path, numbered dashed = ACKs, control, offline